Developing Back-End Apps with Node.js and Express
Module 2 Cheat Sheet: Asynchronous I/O with Callback Program

Package/Method	Description	Code Example
Async-await	We can await promises as long as they are being called inside asynchronous functions.	1. 1 2. 2 3. 3 4. 4 5. 5 6. 6 7. 7 8. 8 1. `const axios = require('axios').default;` 2. `let url = "some remote url"` 3. `async function asyncCall() {` 4. `  console.log('calling');` 5. `  const result = await axios.get(url);` 6. `  console.log(result.data);` 7. `}` 8. `asyncCall();` Copied!
Callback	Callbacks are methods that are passed as parameters. They are invoked within the method to which they are passed as a parameter, conditionally or unconditionally. We use callbacks with a promise to process the response or errors.	1. 1 2. 2 3. 3 4. 4 5. 5 6. 6 1. `//function(res) and function(err) are the anonymous callback functions` 2. `axios.get(url).then(function(res) {` 3. `  console.log(res);` 4. `}).catch(function(err) {` 5. `  console.log(err)` 6. `})` Copied!
Promise	An object that is returned by some methods, representing eventual completion or failure. The code continues to run without getting blocked until the promise is fulfilled or an exception is thrown.	1. 1 2. 2 3. 3 4. 4 5. 5 1. `axios.get(url).then(` 2. `//do something` 3. `).catch(` 4. `//do something` 5. `)` Copied!
Promise use case	Promises are	1. 1 2. 2

	used when the processing time of the function we invoke takes time like remote URL access, I/O operations file reading, etc.	3
		4
		5
		6
		7
		8
		9
		10
		11
		12
		13
		14
		15
		16
		
		`let prompt = require('prompt-sync')();`
		`let fs = require('fs');`
		`const methCall = new Promise((resolve,reject)=>{`
		`  let filename = prompt('What is the name of the file ?');`
		`  try {`
		`  const data = fs.readFileSync(filename, {encoding:'utf8', flag:'r'});`
		`  resolve(data);`
		`  } catch(err) {`
		`  reject(err)`
		`  }`
		`});`
		`console.log(methCall);`
		`methCall.then(`
		`  (data) => console.log(data),`
		`  (err) => console.log("Error reading file")`
		`);`
		
		Copied!

object.on()	It defines an event handler that the framework calls when an event occurs	1
		2
		3
		4
		5
		6
		7
		8
		9
		10
		11
		
		`http.request( options, function(response) {`
		`  let buffer = '' ;`
		`  ...`
		`  response.on('data', function(chunk) {`
		`  buffer += chunk;`
		`  });`
		`  response.on('end', function() {`
		`  console.log(buffer);`
		`  });`
		`}).end();`
		
		Copied!

Callback Hell/The Pyramid of Doom	Nested callbacks stacked below one another and waiting for the previous callback. This creates a pyramid structure that affects the readability and maintainability of the code.	1
		2
		3
		4
		5
		6
		7
		8
		9
		10
		11
		
		`const makeCake = nextStep => {`
		`  buyIngredients(function(shoppingList) {`
		`  combineIngredients(bowl, mixer, function(ingredients){`
		`  bakeCake(oven, pan, function(batter) {`
		`  decorate(icing, function(cake) {`
		`  nextStep(cake);`
		`  });`
		`  });`
		`  });`

```
  });
};
```

Copied!

Axios Request	The axios package handles HTTP requests and returns a promise object.	

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
```

```
const axios = require('axios').default;
const connectToURL=(url)=>{
  const req=axios.get(url);
  console.log(req);
  req.then(resp=>{
  console.log("Fulfilled");
  console.log(resp.data);
  })
  .catch(err=>{
  console.log("Rejected");
  });
}
connectToURL('valid-url')
connectToURL('invalid-url')
```

Copied!

Changelog

Date	Version	Changed by	Change Description
04-07-2022	1.0	Pallavi	Initial version created
18-10-2022	1.1	K Sundararajan	Cheatsheet updated
17-11-2022	1.2	K Sundararajan	IDSN logo updated based on Beta feedback
29-11-2022	1.3	K Sundararajan	Title updated based on Beta feedback

© IBM Corporation 2022. All rights reserved.